

SERVICE ACTUEL · DATA-MODEL

Catalogue, contrats et cycle de vie des datasets

21 capacités analysées

services/data-model · Kotlin 2.4.10 et Spring Boot 4.1.1 sur JVM ; modules domaine et adaptateurs Spring ; PostgreSQL, JPA/JDBC, Flyway ; Kafka et Protobuf ; clients HTTP IAM et Datasphere.

Référentiel métier utilisé par le catalogue, les projets, les producteurs de données et les responsables qualité. Il réserve les identités de datasets, conserve les métadonnées de branches et l'historique des rafraîchissements, gère les liens vers les projets et diffuse les faits nécessaires aux autres services. Il ne lit pas les lignes des tables.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Catalog, Datasets, IAM, Flows, Connectors, Writer, Publisher, Audit, Maintenance, Notifications, Egress, Migration.

Périmètre de lecture

- Les 14 contrôleurs REST, leurs DTO et les familles de commandes/requêtes dataset, label et promotion.
- Agrégats Dataset, DatasetBranch, schéma, refresh, qualité, connecteurs et formats ; repositories de catalogue, labels et suppressions.
- Consommateurs data-collect, data-dev, process-data, data-serve et IAM ; outbox ; runner de migration et tâches planifiées.
- AGENTS.md, registre des contrôles résiduels, propriétés de service, versions Gradle et charts.

Objets et cycle de vie

- Dataset porte l'identité, le nom technique, le projet et les drapeaux reserved/hidden/inProduction/isPublished. Les propriétés affichées principales viennent de la branche métier main. SourceDataset décrit une origine et son éventuelle suppression.
- DatasetBranch est une branche métier persistée dans PostgreSQL, distincte d'une ref physique Iceberg. Elle porte propriétaire, état, qualité, sensibilité, annotations et schéma. DatasetColumn utilise aujourd'hui un UUID métier et un en-tête ; il ne représente pas explicitement un field ID Iceberg.
- DatasetRefreshConfig décrit déclenchement, append/override, schéma MERGE/REPLACE/PERMISSIVE_FIXED/STRICT_FIXED, schedules et retry. DatasetRefreshHistory décrit une exécution ou une opération de schéma avec ses entrées, opérations et logs.
- DatasetProjectPermission porte demande/accord/refus et validation. DatasetMembership porte un abonnement tel que DATASET_QUALITY, distinct d'un droit de lecture.
- Label et SchemaLabel sont deux vocabulaires distincts. Rule/RuleHistory/Compute/ComputedRule sont des projections de qualité reçues de data-serve.
- LabelPolicy est tenue par le service Datasphere optionnel. PendingPromotion, DatasetPromoted et DatasetPromotionHistory suivent des demandes d'effets distants.
- DatasetDeletion conserve l'identité physique et bloque la réutilisation du nom jusqu'au drop. Les tables de miroirs IAM et correspondances servent la migration et la présentation.

Dépendances du service

- data-collect publie les créations/évolutions de source et les états de collecte en Protobuf sur Kafka ; data-model enregistre configuration et refresh.
- data-dev publie versions, branches et sorties de flows ; process-data publie événements de traitement, horaires et résultats. data-model publie dataset, lien projet, schedule et empty vers les consommateurs.
- data-serve publie les règles et leurs résultats ; il peut aussi émettre les corrections de types sur le topic de migration de schéma.
- IAM HTTP décide consultation par lot, édition de projet, gestion de labels et transfert de propriétaire. IAM consomme les faits authz et écrit OpenFGA.
- notification-center reçoit abonnements et notifications, puis alimente la projection locale des memberships.
- Datasphere HTTP optionnel possède les labels de sécurité, les décisions de clearance et l'effet encrypt. PostgreSQL est le stockage métier actuel ; aucun moteur SQL de données n'est embarqué ici.

Fonctionnalités et compatibilité à préserver

DM-01 **Cible non qualifiée**

Réserver une identité et un nom de table, seul ou en lot

COMPOTEMENT OBSERVÉ

La création alloue un UUID et une branche métier main vide en état CREATED dans PostgreSQL. Ce parcours réserve des métadonnées et ne crée pas de snapshot ni de ref physique Iceberg. Le dataset est reserved, hors production et caché lorsqu'il appartient à un projet. Sans projet, sa visibilité catalogue est déjà publiée. Le DTO exige un nom affiché non blanc et un nom technique composé de deux parties imprimables séparées par un point. Le handler normalise la partie table, refuse les collisions dans le projet ou le catalogue et vérifie que le propriétaire est actif. Le lot accepte de 1 à 100 éléments, impose un seul projet et détecte aussi les collisions entre éléments du même lot. Il peut abonner l'auteur aux échecs de collecte. Une attente bornée laisse IAM propager le droit du propriétaire après création ; elle ne transforme jamais une création réussie en refus.

UTILISATEURS ET RÔLES

Auteur de flow ou de collecte, propriétaire, interface catalogue et SDK.

INTERFACES DANS LE CODE

POST /datasets ; POST /datasets/bulk ; GET et POST /datasets/check ; GET /projects/{projectId}/table-name.

DONNÉES ET ÉTATS

Dataset, branche métier main, réservation et namespace PostgreSQL ; miroir utilisateurs/projets ; faits de création.

DÉPENDANCES ET ÉCHANGES

IAM pour le droit sur le projet ; outbox authz ; notifications ; collecte ultérieure qui consomme la réservation.

COMPOTEMENT CIBLE PROPOSÉ

Datasets Rust conserve UUID et alias, avec une commande idempotente de réservation et une contrainte d'unicité transactionnelle. La réservation ne crée aucune édition publiée. Catalog expose les noms et états, IAM garde la décision sur le projet et le propriétaire.

MIGRATION DE CETTE CAPACITÉ

Importer UUID, namespaces, alias et réservations encore utilisées. Conserver la limite de lot et les codes d'erreur via une façade. Remplacer les sujets Keycloak et IDs projet hérités par des IDs IAM stables avec table de correspondance.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un lot qui contient deux noms se normalisant de la même manière est refusé avant toute création. Après une réponse perdue, la même clé de commande retrouve la même réservation. Le propriétaire peut ensuite lire l'objet sans devoir le recréer.

Références de l'analyse

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/CreateDatasetCommandHandler.kt:51](#) État initial, branche, vérifications et création en lot.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetCreateController.kt:124](#) Borne de lot, canonisation et attente de propagation.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/DatasetCreateDTO.kt:22](#) Contraintes du contrat public et propriétaire par défaut.

DM-02 **Cible non qualifiée****Modifier les métadonnées et transférer la propriété****COMPOTEMENT OBSERVÉ**

Le détail assemble propriétaire, origine, description, état, médaillon qualité, sensibilité, branche, planning, labels, volumes, configuration et abonnements de l'utilisateur. PUT permet nom, description, ownerId, qualité, labels et certains réglages de refresh. Un dataset encore produit par un dataflow vivant ne peut pas être modifié par ce parcours. Le transfert effectif du propriétaire demande `can_manage_shares` à IAM avant de vérifier le nouvel utilisateur. Seul le changement du propriétaire de la branche métier main déplace le droit global et retire les anciennes notifications qualité. La sensibilité et le mode de schéma ne sont pas modifiables par ce DTO.

UTILISATEURS ET RÔLES

Propriétaire de dataset, gestionnaire de catalogue, consommateur du détail.

INTERFACES DANS LE CODE

GET `/datasets/{datasetId}` ; PUT `/datasets/{datasetId}` avec `branchName` ; `DatasetDetailQueryHandler` et `UpdateDatasetFromFrontCommand`.

DONNÉES ET ÉTATS

Dataset global et `DatasetBranch` avec propriétaire, dates, description, qualité BRONZE/SILVER/GOLD et sensibilité INTERNAL/RESTRICTED/SENSIBLE.

DÉPENDANCES ET ÉCHANGES

IAM pour transfert ; miroirs utilisateurs ; événements dataset, planning et `owner_changed` ; notifications.

COMPOTEMENT CIBLE PROPOSÉ

Catalog Rust porte les métadonnées descriptives et leur historique de commandes. IAM porte propriété et classification. Datasets lie toute métadonnée technique à une édition ou à un draft explicite. La visibilité, la disponibilité des données et le résultat du dernier run restent des champs distincts.

MIGRATION DE CETTE CAPACITÉ

Reprendre les métadonnées par branche et les alias d'identités. Documenter le fait qu'un null dans le DTO actuel conserve souvent la valeur plutôt que l'effacer. Introduire une version de commande pour éviter les modifications concurrentes perdues.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un changement de description conserve l'édition lue. Un transfert refusé ne révèle pas si le nouvel utilisateur existe et ne modifie aucune ligne. Un transfert autorisé de main retire le droit de l'ancien propriétaire et l'abonnement associé une seule fois.

Références de l'analyse

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/DatasetDetailDTO.kt:26](#) Champs du détail réellement exposés.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/DatasetUpdateDTO.kt:16](#) Champs modifiables et exclusion du mode de schéma.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/UpdateDatasetCommandHandler.kt:522](#) Préconditions, autorisation du transfert et événements.

DM-03 **Cible non qualifiée****Rechercher le catalogue et distinguer découverte et accès aux données****COMPOTEMENT OBSERVÉ**

Trois recherches couvrent le catalogue publié, les datasets d'un projet et le catalogue enrichi de l'état de lien à un projet. Elles filtrent le texte, sensibilité, qualité, état, type et ID d'origine, déclenchement, mode de refresh et labels ; le projet ajoute la diffusion. Les cartes du catalogue restent découvrables même si le lecteur ne peut pas consulter les lignes, afin de permettre une demande d'accès. Le repository impose published, in_production, non reserved et non hidden pour le catalogue. Le tri est une liste de champs admis avec UUID de départage. REPLACE est traduit en OVERRIDE et l'ID d'origine accepte sa forme nue ou préfixée. Une taille non positive signifie actuellement une lecture sans borne pratique.

UTILISATEURS ET RÔLES

Lecteur du catalogue et membre de projet qui cherche ou demande un dataset.

INTERFACES DANS LE CODE

GET /catalog/datasets ; GET /catalog/projects/{projectId}/datasets ; GET /catalog/dataset-with-project-links.

DONNÉES ET ÉTATS

Vue dataset_catalog et projections PostgreSQL des propriétaires, origines, labels et permissions de projet.

DÉPENDANCES ET ÉCHANGES

Gateway IAM pour la route ; correspondance des IDs projet ; aucune lecture de table physique dans ce parcours.

COMPOTEMENT CIBLE PROPOSÉ

Catalog Rust conserve deux décisions séparées, découvrir une carte et lire une édition. Les index restent reconstructibles à partir des commandes métier et des publications approuvées. Les recherches ont des limites explicites et des curseurs stables, sans faire fuiter les schémas protégés.

MIGRATION DE CETTE CAPACITÉ

Rejouer les filtres, normalisations et tris existants sur un corpus comparé. Annoncer le remplacement de `size<=0` par une limite ou une tâche d'extraction. Ne pas masquer les cartes restreintes si le parcours de demande d'accès les exige.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur qui découvre une carte restreinte peut demander son accès mais reçoit un refus sur son schéma et ses lignes. Tous les filtres combinés et leurs totaux correspondent au corpus migré ; les réservations et datasets cachés restent exclus.

Références de l'analyse

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetCatalogController.kt:33](#) Trois contrats de recherche et paramètres.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/SearchCatalogDatasetsQueryHandler.kt:30](#) La découverte publiée ne vaut pas droit de lecture.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/repository/DatasetCatalogRepositoryImpl.kt:36](#) Prédicats de visibilité et requêtes.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/repository/DatasetCatalogRepositoryImpl.kt:320](#) Filtres, alias, pagination et tri.

DM-04 **Cible non qualifiée****Mesurer les usages du catalogue****COMPOTEMENT OBSERVÉ**

Les KPI de datasets retournent un total publié non caché et les plus utilisés, mesurés par le nombre de liens accordés avec des projets publics. Ils ne mesurent pas les requêtes SQL ou les consultations. Le KPI de sources compte les datasets groupés par origine en excluant LOCAL et DATAFLOW. Les prédicats du total ne sont pas identiques à ceux de la recherche du catalogue, qui exclut aussi réservations et hors production.

UTILISATEURS ET RÔLES

Administrateur et tableau de bord du catalogue.

INTERFACES DANS LE CODE

GET /catalog/kpi/datasets ; GET /catalog/kpi/datasources, paramètre `maxResult`.

DONNÉES ET ÉTATS

Agrégations PostgreSQL de dataset, dataset_branch, dataset_permission et project.

DÉPENDANCES ET ÉCHANGES

Gateway sur les routes KPI ; projections des projets publics et sources.

COMPOTEMENT CIBLE PROPOSÉ

Catalog Rust préserve les indicateurs de liens en les nommant précisément. Un indicateur distinct de consommation peut venir des reçus Query et des EXPOSURE déclarées, avec son périmètre et sa durée de conservation.

MIGRATION DE CETTE CAPACITÉ

Reprendre les définitions existantes avant de changer les compteurs. Décider si le total doit suivre les mêmes règles que la recherche. Ne pas transformer un nombre de liens en métrique d'usage réel.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux liens accordés et une demande en attente produisent deux usages de partage. Un lien à un projet privé n'entre pas dans le classement public. Un téléchargement n'augmente pas ce KPI historique.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetCatalogKpiController.kt:23](#) Endpoints KPI.

[monorepo · services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/dataset/repository/DatasetCatalogRepositoryImpl.kt:234](#) Calcul exact des compteurs et classement.

DM-05 **Cible non qualifiée****Lire schémas et métadonnées par lot, et servir le SDK historique****COMPOTEMENT OBSERVÉ**

Le service expose le schéma d'un dataset ou de plusieurs datasets, éventuellement par projet et branche. Sans IDs explicites ni branche, il ne garde que la production. Les résultats de schéma passent un lot de décisions IAM fondées sur dataset, branche et liens accordés ; une réponse mal alignée échoue. POST /datasets/list peut déléguer le contrôle du corps à la gateway lorsque le drapeau de déploiement est activé, sinon interroge IAM. Le SDK dispose d'une liste dédiée /lib/datasets qui renvoie nom affiché, UUID, nom technique et table exprimée en UUID@main. Un ancien alias sélectionné par User-Agent pylib subsiste.

UTILISATEURS ET RÔLES

Interface SQL, éditeur de flow, bibliothèque Python et clients de catalogue.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/schema ; GET /datasets/schema ; POST /datasets/list ; GET /lib/datasets ; alias GET /datasets avec User-Agent pylib.

DONNÉES ET ÉTATS

Colonnes de branche, IDs de labels de colonnes, description et permission calculée ; projection en lecture.

DÉPENDANCES ET ÉCHANGES

IAM batch consultation ; repository de liens accordés ; IDs projet hérités traduits.

COMPOTEMENT CIBLE PROPOSÉ

Datasets Rust fournit un SchemaRef versionné associé à une EditionRef ; Catalog conserve les annotations. La façade SDK garde temporairement les anciennes formes d'identifiant. Chaque collection de résultats filtre les objets autorisés avant de calculer pagination et totaux.

MIGRATION DE CETTE CAPACITÉ

Mapper UUID@main vers une édition publiée résolue au début du run. Migrer les clients vers la route dédiée, puis retirer l'alias User-Agent seulement après inventaire. Tester séparément les deux modes de contrôle du corps.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une requête mélangeant deux datasets autorisés et un interdit ne révèle aucun schéma du troisième. Une décision IAM incomplète ne retourne pas un résultat partiel. Un ancien client peut toujours résoudre les tables de son projet pendant la période de compatibilité.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/DatasetSchemaQueryHandler.kt:72](#) Sélection des branches et filtrage central.

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/DatasetListQueryHandler.kt:26](#) Bascule entre contrôle du corps gateway et IAM.

[monorepo](#) · [services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetForLibController.kt:28](#) Contrat SDK et route de compatibilité.

DM-06 Cible non qualifiée

Synchroniser les branches issues des dataflows

COMPORTEMENT OBSERVÉ

Les événements de data-dev portent les sorties des scripts et les actions SAVE, MERGE et PUBLISH. Ils mettent à jour ou créent les branches métier en PostgreSQL, reprennent propriétaire, qualité, sensibilité et labels, et retirent les sorties devenues orphelines selon la branche et l'état du flow. Une nouvelle branche duplique les colonnes de la branche métier main avec de nouveaux UUID de colonne et sans annotations. inProduction est monotone. Le mode de schéma du flow n'est adopté que lors de la création de la configuration. La suppression d'un flow peut conserver explicitement certains datasets en marquant leur source supprimée.

UTILISATEURS ET RÔLES

Auteur de dataflow, ordonnanceur et consommateurs des événements de versions.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/branches ; DataflowDecisionCommandHandler ; événements data-dev-graph-execution et data-dev-dataflow.

DONNÉES ET ÉTATS

DatasetBranch métier, SourceDataset, configuration, branches retirées et sorties conservées.

DÉPENDANCES ET ÉCHANGES

data-dev fournit le graphe et ses sorties ; process-data confirme les données et schémas.

COMPORTEMENT CIBLE PROPOSÉ

Datasets et Flows Rust partagent des identités stables de sorties, drafts, environnements et éditions. Une branche de travail référence explicitement sa base ; un merge prépare des candidats sous WAP et ne vaut pas publication. Les field IDs Iceberg remplacent la duplication aveugle des UUID de colonnes.

MIGRATION DE CETTE CAPACITÉ

Importer le mapping branche métier vers ref physique et relever les divergences. Conserver les règles de sorties gardées ou supprimées. Couper l'ancien consommateur pour chaque cohorte avant que Rust devienne le seul propriétaire des transitions.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un merge qui retire une sortie de la branche ne supprime pas une édition encore référencée en production. Une nouvelle branche garde sa base vérifiable. Un événement répété ne crée ni deuxième branche ni nouvelle publication.

Références de l'analyse

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/DataflowDecisionCommandHandler.kt:174](#) Actions réellement acceptées et nettoyage des sorties.

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Dataset.kt:104](#) Création et mise à jour des branches et configuration.

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Schema.kt:52](#) Duplication de colonnes et perte volontaire des annotations.

DM-07 **Cible non qualifiée**

Éditer la structure et les annotations de colonnes

COMPOTEMENT OBSERVÉ

PUT schema distingue les annotations seules des renommages, casts et suppressions. Il exige une commande non vide, des IDs de colonnes présents une seule fois et des labels existants. Les noms renommés suivent une grammaire normalisée et restent uniques après toutes les opérations. Les casts suivent une matrice codée ; une branche secondaire ne peut modifier une colonne héritée de la branche métier main. Il faut garder au moins deux colonnes et ne pas supprimer la dernière. Une opération structurelle est refusée si le dernier refresh n'est pas final, sauf s'il est jugé périmé après quinze minutes. Le catalogue écrit immédiatement le schéma souhaité et un historique CREATED ; une annotation seule crée un historique SUCCESS.

UTILISATEURS ET RÔLES

Gestionnaire du dataset ou auteur autorisé à modifier son schéma.

INTERFACES DANS LE CODE

PUT /datasets/{datasetId}/schema ; UpdateSchemaCommand et UpdateColumnCommand.

DONNÉES ET ÉTATS

DatasetColumn, labels, schemaBefore/schemaAfter et historique UPDATE_COLUMN ; événements de traitement ultérieurs.

DÉPENDANCES ET ÉCHANGES

Process-data/data-dev exécutent la modification physique ; data-serve peut réconcilier les types physiques.

COMPOTEMENT CIBLE PROPOSÉ

Datasets Rust sépare ProposedSchema et schéma de l'édition publiée. Writer réalise la transformation sur candidat, Audit contrôle les types et Publisher publie après CAS. Le schéma visible aux lecteurs ne change pas avant publication. Les annotations restent liées aux field IDs.

MIGRATION DE CETTE CAPACITÉ

Importer schémas déclarés et physiques côte à côte puis traiter les écarts. Requalifier la matrice actuelle, notamment les conversions vers flottant, sans la déclarer sans perte par son commentaire. Remplacer le délai de quinze minutes par un lease et une décision de reprise explicites.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un cast qui échoue laisse l'ancien schéma et l'ancienne édition visibles. Un champ renommé conserve son field ID et ses annotations. Deux commandes concurrentes sur la même base aboutissent à un conflit explicite ou à un seul résultat publié.

Références de l'analyse

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/UpdateSchemaCommandHandler.kt:132](#) Validations des opérations de colonne.

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/UpdateSchemaCommandHandler.kt:298](#) Écriture immédiate et historique avant exécution physique.

[monorepo](#) · [services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Schema.kt:9](#) Matrice des casts réellement codée.

DM-08 **Cible non qualifiée**

Conserver les contrats de format et de préparation des entrées

COMPORTEMENT OBSERVÉ

Les configurations de refresh gardent CSV, tableur, JSON, XML et Parquet. Les formats portent encodage, compression NONE/GZIP, MIME, suffixes et présence d'en-tête. CSV ajoute séparateur, commentaire, lignes et colonnes ignorées, trim et format temporel ; tableur ajoute feuille ; JSON et XML ajoutent leur chemin d'extraction. Les préparations comprennent transposition, nom de fichier, URL et date ou date-heure d'import. Les filtres de fichiers portent égalité, exclusion ou sous-chaîne du nom. Ce service conserve et expose ces choix ; il n'en exécute pas les parseurs.

UTILISATEURS ET RÔLES

Utilisateur de collecte et lecteur de la configuration historique.

INTERFACES DANS LE CODE

DatasetConfigDTO ; configurations dans GET /datasets/{datasetId} et dans le détail de refresh ; événements de collecte.

DONNÉES ET ÉTATS

ConfigConnector, FileFormat, Preparation et filtres rattachés à la source/historique.

DÉPENDANCES ET ÉCHANGES

data-collect et process-data produisent les événements et réalisent les lectures/préparations.

COMPORTEMENT CIBLE PROPOSÉ

Connectors et Datasets Rust versionnent un FormatContract utilisable par des workers bornés. Le parsing et les normalisations reçoivent une version de contrat, des limites d'octets et un rapport d'erreurs. Les fichiers originaux peuvent former un FILESET publié distinct de la TABLE interprétée.

MIGRATION DE CETTE CAPACITÉ

Traduire chaque option avec un corpus de fichiers réels anonymisés. Préserver la configuration appliquée au run, pas seulement le réglage courant. Ne pas reprendre comme garantie la constante XLSX non utilisée ou les commentaires sur dates sans qualification.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Le même fichier, contrat et moteur versionné donnent le même schéma et le même compte de lignes. L'encodage invalide, la feuille absente et le dépassement de taille produisent un échec borné sans publication. Les colonnes ajoutées restent traçables.

Références de l'analyse

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Dataset.kt:463` Formats source et filtres déclarés.

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/fileFormat/CsvFileFormat.kt:5` Options CSV.

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/fileFormat/XlsxFileFormat.kt:5` Options tableur.

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/fileFormat/JsonFileFormat.kt:5` Chemin JSON.

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/fileFormat/XmlFileFormat.kt:5` Chemin XML.

`monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Preparation.kt:3` Préparations représentées.

DM-09 **Cible non qualifiée**

Restituer les paramètres des sources et des connecteurs

COMPORTEMENT OBSERVÉ

Le domaine distingue connexions et configurations de collecte. Il représente fichiers locaux, SFTP, dépôt, S3, API, PostgreSQL, MySQL, MSSQL, Oracle, BigQuery, Google Sheets, Jira, Cleyrop, Snowflake, Salesforce, MS Dynamics, SAP S/4HANA, Business Central et SharePoint. Les sources SQL portent table, type et requête personnalisée. Les APIs portent pagination par page, offset, curseur, lien JSON ou Link HTTP. Les quatre connecteurs métier typés gardent entity/select/order/filter/limit et version API, avec service ou company selon le cas. Les DTO ne les réduisent pas à une API générique et les credentials restent dans la couche datasource de data-collect.

UTILISATEURS ET RÔLES

Utilisateur qui inspecte une collecte, support et futur migrateur de Sources.

INTERFACES DANS LE CODE

DatasetConfigDTO.toDTO ; Connector/ConfigConnector ; détail dataset et historique.

DONNÉES ET ÉTATS

Configuration typée dans PostgreSQL, associée au refresh ; origine source et éventuel chemin de fichier.

DÉPENDANCES ET ÉCHANGES

data-collect autoritaire pour les connexions ; protocole Protobuf et mappings DAO/DTO.

COMPORTEMENT CIBLE PROPOSÉ

Connectors Rust possède les références de secrets et les capacités des drivers. Catalog affiche une projection expurgée du contrat exécuté. Le type de source reste indépendant du kind de dataset. Une pagination source n'est jamais utilisée comme checkpoint publié par simple copie.

MIGRATION DE CETTE CAPACITÉ

Reprendre les discriminants et configurations sans exporter de secrets. Vérifier chaque round-trip DAO/Protobuf/DTO. Porter les curseurs durables vers origin.* du candidat et ne les acquitter qu'après publication.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une configuration SQL personnalisée ou OData conserve tous ses champs après migration. Le détail historique ne révèle aucune valeur secrète. Une reprise de pagination ne saute aucune page après une publication refusée.

Références de l'analyse

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/connector/Connector.kt:9](#) Familles de connecteurs actuelles.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/configconnector/PaginationConfig.kt:13](#) Six modes de pagination.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/config/DatasetConfigDTO.kt:38](#) Correspondance exhaustive des DTO.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/config/DatasetConfigDTO.kt:114](#) Conservation des champs typés et séparation des credentials.

DM-10 **Cible non qualifiée**

Suivre les collectes, traitements et historiques de refresh

COMPORTEMENT OBSERVÉ

Le service agrège les événements de collecte et de traitement dans un historique par dataset et branche. Il conserve numéro d'exécution, auteur, déclenchement, append/override, état, opérations ingestion/process, dates, volumes, logs et entrées fichiers ou tables avec erreurs. Il calcule les durées et associe le compute qualité au refresh. Les routes de détail et logs vérifient que le refresh appartient au dataset. Les statuts terminaux et les opérations de schéma suivent des chemins séparés pour ne pas écraser un résultat final par un événement running tardif.

UTILISATEURS ET RÔLES

Auteur, exploitant, support et utilisateur du détail dataset.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/refresh ; GET /datasets/{datasetId}/refresh/{refreshId} ; GET /datasets/{datasetId}/refresh/{refreshId}/logs/{logType}.

DONNÉES ET ÉTATS

DatasetRefreshHistory, DataInputFileRefreshRecord, DataInputTableRefreshRecord, Ingestion/Process/UpdateSchema et logs PostgreSQL.

DÉPENDANCES ET ÉCHANGES

Consommateurs data-collect/process-data ; données qualité reçues de data-serve ; notifications d'échec.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust conserve les runs et leurs tentatives ; Datasets expose leur relation avec candidats et éditions. Audit garde les preuves immuables. Le checkpoint publié provient de l'édition Iceberg autoritaire, même si une projection de run a pris du retard.

MIGRATION DE CETTE CAPACITÉ

Conserver les anciens IDs de refresh en références de migration et importer logs avec accès contrôlé. Rejouer les événements hors ordre pour vérifier les transitions. Mapper volumes et durées sans inventer de snapshot pour les échecs historiques.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un événement running tardif ne rouvre pas un run terminal. Le détail d'un refresh d'un autre dataset retourne un introuvable. Après perte de la projection, le dernier checkpoint se reconstruit de l'édition publiée sans réémission d'effet.

Références de l'analyse

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/DatasetRefresh.kt:19](#) Modèle des historiques et entrées.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/RefreshHistoryRecord.kt:38](#) Protection des états terminaux de process.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/DatasetRefreshHistoryDetailQueryHandler.kt:46](#) Lien parent/enfant, durées et qualité.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/OperatorLogsQueryHandler.kt:62](#) Restriction des logs au couple dataset/refresh.

DM-11 **Cible non qualifiée**

Régler le déclenchement et les reprises planifiées

COMPORTEMENT OBSERVÉ

Le modèle conserve déclenchements manuel, planifié, récurrent ou événements dataset/dataflow/fichier, avec schedules, fuseau et RetryPolicy. La mise à jour depuis le catalogue ne prend en charge que MANUAL ou RECURRENT et ne modifie le planning que pour une source vivante autre que DATAFLOW et LOCAL. Elle émet DatasetScheduleUpdatedEvent ; ce service n'exécute pas lui-même les collectes planifiées. La configuration du schéma reste séparée de cette modification.

UTILISATEURS ET RÔLES

Propriétaire qui règle le refresh et ordonnanceur consommateur.

INTERFACES DANS LE CODE

PUT /datasets/{datasetId} avec refreshConfig ; DatasetScheduleUpdatedEvent ; SynchronizedRefreshCommandHandler.

DONNÉES ET ÉTATS

DatasetRefreshConfig, Scheduler et RetryPolicy ; événements de planning.

DÉPENDANCES ET ÉCHANGES

process-data/data-collect pour déclenchement et retours ; zone horaire du contrat.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust devient l'unique propriétaire du schedule, de ses occurrences, reprises et arrêts. Catalog conserve une projection lisible. Chaque occurrence lance un run borné avec ID stable, deadline, budget de retry et checkpoint publié de départ.

MIGRATION DE CETTE CAPACITÉ

Importer cron, timezone et retry en préservant l'heure locale et les passages heure été/hiver. Pour chaque cohorte, désactiver le propriétaire ancien avant d'activer Rust. Décider explicitement le traitement des occurrences manquées.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une occurrence à la transition de fuseau suit une règle documentée sans double publication. Un redémarrage d'ordonnanceur retrouve le même ID d'occurrence. Modifier les labels seuls ne change pas le prochain déclenchement.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/DatasetRefreshConfig.kt:10](#) Schedules et paramètres de reprise.

[monorepo · services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/dto/DatasetDTO.kt:183](#) Déclenchements exposés et mises à jour admises.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/UpdateDatasetCommandHandler.kt:582](#) Sources réellement modifiables et événement de planning.

DM-12 **Cible non qualifiée**

Rendre visible un dataset de projet

COMPORTEMENT OBSERVÉ

publish refuse un dataset caché puis met seulement isPublished à true, sauvegarde le dataset et émet les faits de visibilité. Il n'effectue aucun commit Iceberg, audit de candidat ou déplacement de ref. unhide est une action distincte. Les données peuvent donc avoir été produites avant cette publication de catalogue. Les permissions et affichages actuels s'appuient sur cette distinction projet privé, projet publié et catalogue global.

UTILISATEURS ET RÔLES

Gestionnaire autorisé à éditer le dataset et lecteurs du catalogue.

INTERFACES DANS LE CODE

POST /datasets/{datasetId}/publish ; POST /datasets/{datasetId}/unhide.

DONNÉES ET ÉTATS

Booléens hidden, inProduction, isPublished et faits authz ; dernier refresh associé au message.

DÉPENDANCES ET ÉCHANGES

IAM/gateway pour can_edit ; événement dataset et dataset.published.

COMPORTEMENT CIBLE PROPOSÉ

Catalog Rust nomme cette opération diffusion. Publisher Rust possède séparément la publication d'une édition approuvée. Une édition reçoit origin.* au summary du candidat, une preuve d'audit durable et une référence approuvée ; la diffusion ne remplace aucun de ces contrats.

MIGRATION DE CETTE CAPACITÉ

Conserver une façade /publish qui préserve d'abord la diffusion historique. Créer une commande séparée de publication de données et migrer les usages un par un. Ne jamais transformer rétroactivement tous les snapshots physiques en publications approuvées.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Publier la diffusion d'un dataset ne modifie ni snapshot ni checkpoint. Un candidat refusé par Audit ne devient pas lisible parce que isPublished est vrai. L'historique utilisateur ne contient que les éditions approuvées.

Références de l'analyse

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/PublishProjectDatasetCommandHandler.kt:58](#) La publication actuelle ne fait que modifier le référentiel et émettre des événements.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Dataset.kt:302](#) Implémentation des drapeaux publish et unhide.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetCreateController.kt:252](#) Actions REST distinctes.

DM-13 **Cible non qualifiée**

Demander, accorder et retirer un accès de projet

COMPORTEMENT OBSERVÉ

Un dataset de projet non publié ne peut pas être lié ailleurs. Pour INTERNAL, la commande vérifie la politique de labels puis accorde immédiatement le lien. Pour RESTRICTED ou SENSIBLE, elle crée une demande avec auteur et commentaire, notifie et abonne le demandeur à la réponse. Accepter enregistre validateur, date et justification, revalide les labels et émet le fait d'accès. Refuser un lien déjà accordé révoque l'alias de projet, sauf le projet propriétaire. La route de retrait exige une justification non blanche, vérifie l'appartenance permission/dataset et n'accepte que l'état ACCEPTED. La liste /access ne montre que cet état, alors que les liens INTERNAL automatiques emploient un autre état accordé.

UTILISATEURS ET RÔLES

Demandeur membre de projet, responsable du dataset, administrateur de partage.

INTERFACES DANS LE CODE

POST /datasets/{datasetId}/projects/{projectId} ; GET /datasets/permissions ; POST /datasets/permissions/{permissionId}/{permission} ; GET /datasets/{datasetId}/permissions/projects ; GET /datasets/{datasetId}/access ; POST /datasets/{datasetId}/access/{permissionId}/withdraw.

DONNÉES ET ÉTATS

DatasetProjectPermission, CREATED/REQUESTED/ACCEPTED/REFUSED/LINKED et justifications ; tuple IAM seulement pour droits accordés.

DÉPENDANCES ET ÉCHANGES

Policy Datasphere, utilisateurs/projets, notification-center, faits authz vers IAM et liens vers data-serve.

COMPORTEMENT CIBLE PROPOSÉ

Catalog et IAM Rust conservent demande, approbation et révocation comme commandes auditées. L'accès porte une portée claire, dataset ou édition, et une politique versionnée. La révocation doit invalider les prochaines pages et téléchargements selon le contrat de lecture.

MIGRATION DE CETTE CAPACITÉ

Importer les états et rattachements sans accorder les demandes en attente. Revoir l'écart entre lien automatique et liste/retrait des accès pour offrir un retrait cohérent. Préserver le projet propriétaire comme relation distincte.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une demande en attente ne permet jamais une lecture. Une acceptation répétée ne crée pas deux liens. Le retrait d'un partage bloque la prochaine page et laisse le projet propriétaire autorisé. La permission d'un autre dataset reste introuvable.

Références de l'analyse

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/CreateDatasetLinkToProjectCommandHandler.kt:140](#)
Différence entre partage immédiat et demande.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/AcceptDatasetLinkToProjectCommandHandler.kt:72](#)
Acceptation et révocation des liens accordés.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetAccessController.kt:35](#) Filtre de liste et conditions de retrait.

Administrer les labels de datasets et de colonnes

COMPORTEMENT OBSERVÉ

Les labels de dataset portent nom et couleur ; la création refuse un nom existant sans tenir compte de la casse. Leur suppression retire le label des branches avant de supprimer la définition et diffuse un événement. Les labels de colonnes sont une entité séparée, avec nom/couleur non blancs, utilisée par UpdateSchema. Le code des handlers create et update de SchemaLabel lève AlreadyExists lorsque getByName retourne null. Le repository renvoie bien null pour un nom absent. Cette condition paraît inversée ; sa conséquence est à reproduire, elle ne doit pas être décrite comme une unicité correcte. Les contrôleurs demandent actuellement à IAM le droit de gérer les labels.

UTILISATEURS ET RÔLES

Administrateur des vocabulaires et éditeur de métadonnées.

INTERFACES DANS LE CODE

GET/POST /labels ; PUT/DELETE /labels/{labelId} ; GET/POST /schema/labels ; PUT/DELETE /schema/labels/{schemaLabelId}.

DONNÉES ET ÉTATS

Label, SchemaLabel, liens aux branches et colonnes ; événements de labels pour data-serve.

DÉPENDANCES ET ÉCHANGES

IAM label management ; repositories et outbox des labels dataset.

COMPORTEMENT CIBLE PROPOSÉ

Catalog Rust garde deux usages explicites, tags de dataset et annotations de champs, avec IDs stables, unicité normalisée et suppression contrôlée. Ces tags descriptifs restent distincts des politiques de sécurité et des classifications IAM.

MIGRATION DE CETTE CAPACITÉ

Importer les IDs et références ; définir la normalisation du nom. Corriger le comportement SchemaLabel avec un ticket vérifiable avant de figer la compatibilité. Conserver l'historique des annotations sur éditions anciennes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer un nouveau label de colonne réussit ; un doublon est refusé selon la règle documentée. Supprimer un label courant ne modifie pas les preuves historiques d'une édition. Deux créations concurrentes ne produisent pas deux définitions équivalentes.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/label/command/CreateLabelCommandHandler.kt:24](#) Unicité du label dataset.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/label/command/DeleteLabelCommandHandler.kt:25](#) Retrait des références de branches.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/label/command/CreateSchemaLabelCommandHandler.kt:25](#) Condition d'existence observée.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/label/command/UpdateSchemaLabelCommandHandler.kt:26](#) Même condition dans update.

[monorepo · services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/label/repository/SchemaLabelRepositoryImpl.kt:27](#) Sémantique effective de getByName.

DM-15 **Cible non qualifiée**

Présenter la qualité et les abonnements de suivi

COMPORTEMENT OBSERVÉ

data-model reçoit les définitions et résultats qualité de data-serve, puis expose le dernier compute par dataset et les résultats associés aux refreshs. Le tableau global sélectionne ALL, OWNER, PROJECTS, CATALOG ou MEMBERSHIP, peut combiner catalogue et projets, et filtre qualité, sensibilité et état. Il autorise les candidats par IAM avant de calculer les stats et ne nomme que les projets accessibles. Il distingue dataset avec règles, succès, erreurs et niveaux CRITICAL/MAJOR/MINOR. Les memberships DATASET_QUALITY viennent aussi des événements de notification ; ils représentent une souscription de suivi.

UTILISATEURS ET RÔLES

Responsable qualité, propriétaire et utilisateur abonné aux alertes.

INTERFACES DANS LE CODE

GET /projects/{projectId}/datasets/monitoring ; POST /quality/monitoring ; DataServeConsumer ; NotificationCenterEventConsumer.

DONNÉES ET ÉTATS

Projections Rule, RuleHistory, Compute, ComputedRule et vue de monitoring ; DatasetMembership.

DÉPENDANCES ET ÉCHANGES

data-serve produit règles/calculs ; notification-center produit abonnements ; IAM filtre consultation et projets.

COMPORTEMENT CIBLE PROPOSÉ

Audit Rust possède les exécutions de règles liées aux candidats et aux éditions ; Catalog en affiche une projection. Notifications possède les préférences de suivi. Une mesure peut bloquer Publisher selon une politique explicite, ce que ce tableau actuel ne garantit pas.

MIGRATION DE CETTE CAPACITÉ

Reprendre définitions, versions et résultats en conservant leur provenance d'exécution. Mapper les abonnements sans les convertir en autorisations. Comparer les agrégats pour chaque catégorie et documenter le statut des résultats historiques sans snapshot.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un dataset interdit ne contribue ni aux items ni aux compteurs personnels. Une règle échouée avant publication laisse l'édition antérieure lisible. Le dernier compute affiché indique la version de règle et l'édition effectivement contrôlée.

Références de l'analyse

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/DatasetQualityController.kt:40](#) Les deux API de monitoring.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/DatasetQualityMonitoringQueryHandler.kt:125](#) Sélection, autorisation et statistiques.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/query/DatasetProjectRulesQueryHandler.kt:86](#) Projection de qualité par projet.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/event/consumer/DataServeConsumer.kt:23](#) Réception des règles et computes.

DM-16 **Cible non qualifiée**

Vérifier les politiques de labels et lire les fichiers SPIF

COMPORTEMENT OBSERVÉ

Une politique est un triplet `policy/classification/category` lié au dataset. Sa modification appelle Datasphere, puis inscrit une promotion en attente. Le contrôle de partage envoie les emails des membres du projet à `/access/check` et refuse les sujets DENY. Une réponse absente du client actuel devient une liste vide, donc aucune interdiction ; la couverture complète des sujets n'est pas vérifiée ici. Si Datasphere est désactivé, l'implémentation `no-op` permet les contrôles et ne fournit pas de politique. Le proxy SPIF exige une URI HTTP(S) absolue et un préfixe autorisé à frontière de chemin avant le GET sortant.

UTILISATEURS ET RÔLES

Administrateur de politique, gestionnaire de partage et lecteur du label de sécurité.

INTERFACES DANS LE CODE

PUT/GET `/datasets/{datasetId}/label/policy` ; GET `/datasets/{datasetId}/projects/{projectId}/label/policy/check` ; PUT `/spif-proxy/files`.

DONNÉES ET ÉTATS

Policy autoritaire externe ; PendingPromotion local ; décisions PERMIT/DENY.

DÉPENDANCES ET ÉCHANGES

Datasphere HTTP `/labels`, `/access/check` et fichier SPIF externe autorisé ; miroir des membres de projet.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust décide sur une politique versionnée et une liste complète de sujets. Catalog garde les références et l'état. Egress centralise le fetch SPIF avec contrôle de destination et de taille ; indisponibilité ou décision incomplète bloque l'opération dépendante.

MIGRATION DE CETTE CAPACITÉ

Inventorier les tenants qui utilisent Datasphere et ceux qui utilisent le `no-op`. Porter le contrat de classification sans inventer de chiffrement. Remplacer l'interprétation permissive des réponses vides par un refus explicite qualifié avec les clients.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un service de politique indisponible ou qui omet un membre bloque le partage. Une URL dont l'hôte imite un préfixe admis est refusée avant l'appel réseau. Une politique modifiée conserve sa version et l'identité de son auteur.

Références de l'analyse

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/promotion/command/LabelPolicyCommandHandler.kt:53](#) Écriture externe puis promotion en attente.

[monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/promotion/query/CheckLabelPolicyQueryHandler.kt:36](#) Décision sur tous les emails du miroir.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/promotion/service/LabelPolicyServiceRestImpl.kt:151](#) Décodage des décisions, valeur vide et appels.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/promotion/service/LabelPolicyServiceRestImpl.kt:217](#) Garde URL SPIF et `no-op` conditionnel.

DM-17 **Cible non qualifiée****Promouvoir sous politique et suivre les demandes de chiffrement****COMPORTEMENT OBSERVÉ**

L'action encrypt vérifie qu'une politique existe, marque le dataset comme promoted puis demande à Datasphere /labels/encrypt sur le nom technique et le dataset. Elle prend le dernier refresh réussi et conserve statut/log de promotion lié à ce refresh. Les datasets marqués sont retraités lors de refreshs ultérieurs. Une tâche périodique traite les changements de label lorsque la politique distante correspond à celle en attente. Sans historique réussi, le handler journalise l'absence mais garde le flag promoted. Ce code ne prouve ni algorithme de chiffrement ni atomicité physique.

UTILISATEURS ET RÔLES

Administrateur de promotion et exploitant qui suit son résultat.

INTERFACES DANS LE CODE

POST/GET /datasets/{datasetId}/label/encrypt ; GET /datasets/{datasetId}/history/{datasetRefreshHistoryId}/encrypt ; AsyncPromoteServiceImpl.

DONNÉES ET ÉTATS

DatasetPromoted, PendingPromotion et DatasetPromotionHistory avec SUCCESS/FAILURE.

DÉPENDANCES ET ÉCHANGES

Datasphere /labels/encrypt ; historique de refresh et tâche périodique locale.

COMPORTEMENT CIBLE PROPOSÉ

Egress et Audit Rust représentent cette opération comme un effet idempotent consommant une EditionRef publiée, avec reçu, politique et preuve. Si un chiffrement change les données ou artefacts de référence, il produit un nouveau candidat sous Writer/Publisher plutôt qu'un alias de publication opaque.

MIGRATION DE CETTE CAPACITÉ

Vérifier le contrat effectif du service distant, les données chiffrées, les lecteurs et les reprises. Importer les historiques comme reçus hérités non qualifiés. Remplacer le simple booléen promoted par un état d'effet demandé/réussi/échoué.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une demande sans édition publiée est refusée explicitement. Après réponse HTTP perdue, le retry retrouve le même reçu et ne chiffre pas deux fois. L'interface n'annonce un succès que lorsque le reçu de l'édition et de la politique attendues existe.

Références de l'analyse

monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/promotion/command/LabelPolicyPromoteCommandHandler.kt:68 Effet externe et historique.

monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/promotion/command/LabelPolicyPromoteCommandHandler.kt:98 Flag avant appel et absence possible de refresh.

monorepo · services/data-model/app/data-model-domain/src/main/kotlin/com/cleyrop/data/model/domain/promotion/command/LabelPolicyPromoteCommandHandler.kt:118 Traitement des relabels.

monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/promotion/service/AsyncPromoteServiceImpl.kt:12 Tâche périodique.

DM-18 **Cible non qualifiée**

Vider, supprimer et libérer les noms sans réexposer des données

COMPORTEMENT OBSERVÉ

Empty vérifie dataset et branche puis émet un événement ; il ne vide pas lui-même la table. Delete refuse une sortie de dataflow dont la source est encore vivante. La suppression habituelle conserve un tombstone DatasetDeletion, le nom technique et les tuples IAM jusqu'au succès du job DATASET_DELETE déclenché côté data-dev. Un échec ou timeout relance l'événement avec nombre d'essais borné ; ABANDONED conserve le nom pour éviter qu'une nouvelle identité lise les anciennes lignes. Les suppressions de projet utilisent toutefois un chemin sans attente de drop. Une tâche libère aussi les réservations sans type trop anciennes.

UTILISATEURS ET RÔLES

Gestionnaire, suppression de projet ou flow, opérateur de reprise.

INTERFACES DANS LE CODE

DELETE /datasets/{datasetId} ; DELETE /datasets/{datasetId}/reserved ; POST /datasets/{datasetId}/empty ; DatasetDropSucceededCommand/FailedCommand.

DONNÉES ET ÉTATS

DatasetDeletion PENDING/FAILED/ABANDONED, branches, nom réservé et faits IAM différés.

DÉPENDANCES ET ÉCHANGES

data-dev/process-data pour drop physique ; événements Kafka et tâches de retry/rétention.

COMPORTEMENT CIBLE PROPOSÉ

Datasets Rust garde le tombstone métier. Maintenance exécute un plan audité pour vider, expirer ou supprimer après vérification de toutes les racines de rétention. Writer/Publisher publie une édition vide sous WAP si nécessaire ; aucun worker n'a le droit REST générique de drop.

MIGRATION DE CETTE CAPACITÉ

Importer toutes les suppressions pendantes et réservations avant de réouvrir les noms. Reprendre le cas projet supprimé avec une autorité Maintenance dédiée. Séparer effacement visible, fin d'accès et GC des octets.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un drop échoué ne libère jamais le nom. Une publication conservée, un run actif ou une release multi-sorties empêche le GC. Une confirmation répétée de drop ne réémet pas les mêmes effets. Empty produit une édition vide approuvée avant de modifier les lectures.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/EmptyDatasetCommandHandler.kt:29](#) Empty est une demande asynchrone.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/DeleteDatasetCommandHandler.kt:130](#) Tombstone et conservation des tuples.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/DeleteDatasetCommandHandler.kt:211](#) Exception de suppression de projet.

[monorepo · services/data-model/app/data-model-](#)

[app/src/main/kotlin/com/cleyrop/data/model/app/dataset/service/DatasetDeletionRetryService.kt:56](#) Retry borné et abandon sûr.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/DatasetDropCommandHandler.kt:28](#) Finalisation idempotente de la suppression.

DM-19 **Cible non qualifiée**

Fournir une liaison de table avant la projection de serving

COMPORTEMENT OBSERVÉ

Le protocole interne retourne datasetId, branche métier, techName, projet, propriétaire, reserved et hasSchema. Il lit la réservation autoritaire avant qu'une première ingestion existe dans data-serve. Si le dataset a disparu, il peut répondre à partir du tombstone en ajoutant deleting. La branche doit exister dans le dataset ou le tombstone. Le contrôleur exige le JWT du compte de service data-serve, en plus du contrôle gateway, et refuse un simple utilisateur du même client OAuth.

UTILISATEURS ET RÔLES

data-serve et protocole de capacité pour jobs de lecture/écriture/suppression.

INTERFACES DANS LE CODE

GET /internal/execution-datasets/{datasetId}?branch=... ; ExecutionDatasetBindingDTO.

DONNÉES ET ÉTATS

Réservation active ou DatasetDeletion autoritaire ; aucune lecture des octets.

DÉPENDANCES ET ÉCHANGES

IAM/gateway et identité de service ; futur consommateur de capacité.

COMPORTEMENT CIBLE PROPOSÉ

Datasets Rust fournit un Binding immuable signé ou référencé qui lie dataset UUID, table UUID Iceberg, namespace, branche, schéma, spec, action, run et époque. Writer vérifie ce contrat et médie staging ; Publisher médie publication. Les CAS et contrôles d'ascendance ne remplacent pas un fencing natif Hemera à construire.

MIGRATION DE CETTE CAPACITÉ

Conserver le chemin compatible pour les exécutions anciennes, mais ne pas réutiliser techName seul comme autorité. Capturer UUID de table et références lors du basculement. Définir un Binding de suppression indépendant du projet déjà effacé.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un worker muni d'un binding expiré ou d'une époque précédente ne peut modifier ni supprimer la table. Renommer un alias ne change pas la table liée. Un compte humain du client data-serve est refusé par le protocole interne.

Références de l'analyse

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/ExecutionDatasetBindingController.kt:16](#) Payload du protocole.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/ExecutionDatasetBindingController.kt:34](#) Vérification du compte de service et branche.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/controller/ExecutionDatasetBindingController.kt:59](#) Résolution à partir du tombstone.

DM-20 **Cible non qualifiée**

Réconcilier les identités et publier les faits d'autorisation

COMPORTEMENT OBSERVÉ

Des miroirs utilisateurs/projets/membres sont alimentés par les anciens événements et, sous drapeaux, par IAM. Les IDs externes des projets et sujets utilisateur sont traduits aux frontières. Les événements IAM couvrent création, mise à jour, suppression et offboarding avec remplacement de propriétaires ; les membres arrivés avant leurs parents passent par une file persistée et un drainage. Les faits dataset créé, publié, reclassifié, propriétaire ou projet changé et supprimé partent vers IAM via outbox. IAM reste l'écrivain OpenFGA. Les drapeaux de migration et de traitement destructif ne constituent pas une preuve de bascule dans un environnement donné.

UTILISATEURS ET RÔLES

IAM, administrateur de migration, responsables des datasets réattribués.

INTERFACES DANS LE CODE

IamEventConsumer ; ProjectIdResolver ; UserIdResolver ; DatasetAuthzEventHandler ; topics IAM et data-model.authz.events.

DONNÉES ET ÉTATS

Miroirs user/project/member, tables de correspondance, pending members, high-water marks et outbox.

DÉPENDANCES ET ÉCHANGES

IAM et anciennes sources benutzer/data-dev ; OpenFGA écrit uniquement par IAM ; notification-center pour transfert.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust possède les identités canoniques ; Catalog consomme des événements ordonnés et dédupliqués pour sa présentation. Les changements de propriétaire et de classification ont une révision et un reçu. Un retard de projection ne permet pas d'autoriser avec une identité périmée.

MIGRATION DE CETTE CAPACITÉ

Importer les correspondances au lieu de supposer IAM UUID égal au subject. Drainer les pending members et prouver la parité avant coupure. Rejouer les faits sur un groupe isolé, puis désactiver les consommateurs hérités cohorte par cohorte.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un offboarding réattribue toutes les branches et n'accorde rien au compte supprimé. Rejouer le journal ne double ni membres ni droits. Un membre reçu avant son projet devient visible après création sans intervention manuelle ni perte d'événement.

Références de l'analyse

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/event/consumer/IamEventConsumer.kt:47](#) Activation et consommation des événements IAM.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/event/consumer/iam/IamUserEventHandler.kt:42](#) Création, suppression et offboarding.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/service/ProjectIdResolver.kt:95](#) Traductions de clés aux frontières.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/dataset/event/consumer/iam/IamPendingMemberDrain.kt:90](#) Drainage des membres différés.

[monorepo · services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/common/outbox/OutboxPublisher.kt:13](#) Contrat de persistance des messages.

DM-21 **Cible non qualifiée**

Ajouter éditions, lignage et collections sans confondre les autorités

COMPORTEMENT OBSERVÉ

Le modèle observé conserve une origine source, les branches, les sorties de flows et des historiques de refresh. Aucun contrôleur dédié aux collections, aucun graphe de lignage par édition et aucune entité Edition avec table UUID, snapshot, schema/spec et preuve d'audit n'ont été trouvés dans ce service. Les types actuels décrivent des connecteurs ou DATAFLOW ; ils ne sont pas les huit kinds de datasets de la cible. Cette fiche décrit donc un besoin cible, pas une capacité livrée.

UTILISATEURS ET RÔLES

Équipe Catalog/Datasets, utilisateurs qui organisent et tracent les données.

INTERFACES DANS LE CODE

Existant : Dataset, SourceDataset, DatasetRefreshHistory et DataflowDecision. API cible de collections/éditions/lignage à spécifier, aucun chemin inventé.

DONNÉES ET ÉTATS

Actuel : PostgreSQL métier et provenance partielle. Cible : commandes/drafts PG et éditions Iceberg autoritaires, index Catalog reconstruisible.

DÉPENDANCES ET ÉCHANGES

Flows fournit entrées et sorties ; Writer crée les candidats ; Audit produit la preuve ; Publisher approuve ; Query épingle les éditions.

COMPORTEMENT CIBLE PROPOSÉ

Datasets Rust définit TABLE, INCREMENTAL, VIEW, FILESET, MODEL, EMBEDDINGS, FEATURESET et STREAM. Chaque kind publie une édition Iceberg avec origin.* dès le candidat et références d'artefacts immuables. Catalog propose des collections de références, sans recopier données ni droits. Le lignage relie éditions d'entrée, contrat, run et édition de sortie. Une release multi-sorties est elle-même une édition avec checkpoint commun et participants, y compris inchangés.

MIGRATION DE CETTE CAPACITÉ

Importer la provenance ancienne avec un niveau de complétude déclaré. Constituer l'historique approuvé sans exposer tous les snapshots techniques. Ne pas déduire de checkpoint depuis un historique de commandes PG. Introduire collections et nouveaux kinds progressivement derrière des contrats versionnés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Perdre l'index Catalog permet de reconstruire les éditions et le checkpoint depuis Iceberg. Une collection ne confère aucun droit supplémentaire. Une release visible ne contient pas de mélange de versions ; ses lecteurs, ack et GC utilisent le même manifeste. STREAM passe aussi par WAP après journal durable.

Références de l'analyse

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/Dataset.kt:66](#) Le dataset actuel ne contient pas de référence d'édition.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/aggregate/DatasetRefresh.kt:28](#) Le refresh historique n'est pas une autorité de snapshot.

[monorepo · services/data-model/app/data-model-](#)

[domain/src/main/kotlin/com/cleyrop/data/model/domain/dataset/command/DataflowDecisionCommandHandler.kt:73](#) Sorties de flows présentes, sans lignage complet par édition.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Conserver les IDs métier, alias, descriptions, collections, demandes de partage, définitions de règles, commandes et drafts dans PostgreSQL avec révisions et idempotency keys. IAM reste seul arbitre des droits.
- Définir EditionRef et DatasetContract avec kind, table UUID, snapshot/ref approuvée, schema/spec, inputs, run, policy et provenance. Résoudre une édition publiée est une opération distincte de la recherche du catalogue.
- Indexer les publications approuvées et les manifests de releases multi-sorties. Une projection peut être en retard ; elle ne peut déplacer le checkpoint publié ni autoriser le GC.
- Transformer la promotion/chiffrement en commande d'effet munie d'un reçu. Conserver les erreurs et reprises sans convertir une demande en succès.

Plan de données proposé

- Writer Rust de confiance valide les bindings et commit le staging dans Lakekeeper. Les workers fournissent fichiers préparés et métadonnées bornées ; ils ne reçoivent pas table.modify générique qui autorise aussi commit et drop.
- Publisher Rust vérifie candidat, droits, preuve post-candidat durable, UUID/refs/schema/spec et ascendance avant publication. Le fencing Hemera, la confidentialité des branches et les races de droits sont des travaux explicites, pas des garanties du CAS seul.
- Écrire origin.system/cursor/state/provenance dans le summary du candidat. Lier ensuite la preuve immuable à l'identifiant préalloué sans prétendre que le fast-forward réécrit ce summary.
- Soumettre compaction, schéma structurel et édition vide à WAP. Maintenance expire les snapshots et objets via plan audité, racines de rétention et relecture après délai de sécurité.

Découpage du core Rust

- Proposition catalog-core : identités de dataset, aliases, métadonnées, recherche, collections et sérialisation des DTO compatibles.
- Proposition dataset-contracts : huit kinds, schémas, field IDs, formats, Binding, EditionRef, PreparedWrite, PublicationReceipt et ReleaseManifest.
- Proposition catalog-projector : projections depuis événements métier et éditions approuvées ; déduplication, version par agrégat, resynchronisation et métriques de retard.
- Proposition dataset-lifecycle : réservation, partage, tombstones et demandes de maintenance avec transactions communes pour état et outbox.
- Proposition migration-adapters : lecteurs PG/Protobuf hérités, mapping IAM, contrôle de parité et adoption des tables existantes sans publication fictive.

Identité, autorisation et isolation

- Les décisions de route sont centralisées dans APISIX/IAM/OpenFGA. Le code conserve des contrôles portant sur des cibles cachées dans le corps, les lots, le transfert d'owner et la liaison permission/dataset. Ne pas déduire de l'AGENTS qu'aucun appel IAM n'existe dans le pod.

- La découverte de cartes publiées est volontairement plus large que l'accès au détail, au schéma et aux octets. Porter cette distinction dans Catalog Rust et les outils MCP.
- La confiance X-Userinfo dépend d'un chemin réseau contrôlé. Le protocole execution-datasets impose aussi une identité de service précise. Les références de secrets de connecteurs ne doivent pas entrer dans origin.* ni les preuves publiques.
- Le périmètre de tenant visible ici repose sur le déploiement et IAM ; les agrégats examinés n'ont pas un tenantId universel. Un service Rust mutualisé doit ajouter une clé de tenant aux commandes, index, caches, objets et décisions.
- Points à corriger/qualifier : réponse de clearance vide interprétée sans DENY ; vérification inversée de SchemaLabel ; contrôle des données potentiellement sensibles dans logs et configurations historiques.

Exploitation et reprise

- Le chart prévoit Deployment, Service, HPA optionnel, PVC optionnel, NetworkPolicy APISIX et PodMonitor. L'activation du PVC force un seul replica et une stratégie de recréation selon le contrat des values. Ce rapport ne prouve aucun déploiement actif.
- PostgreSQL et Kafka sont nécessaires aux commandes et projections. Les migrations Flyway acceptent l'out-of-order ; les migrations de code ont leurs propres lignes de suivi. Le runner data-model actuel lève une exception en cas d'échec, contrairement à la mention exit 0 encore présente dans AGENTS.md.
- Le relay outbox publie au moins une fois, conserve erreurs/essais et purge les messages envoyés selon rétention. La transaction de l'outbox ne prouve l'atomicité état/événement que si l'appelant possède une transaction englobante ; contrôler chaque parcours lors du portage.
- Tâches distinctes : libération des réservations, retry des suppressions, relabel/promotion, drainage des membres IAM et rétention outbox. Toute réplication exige un claim ou verrou distribué pour les tâches mutantes.
- Sauvegarder les commandes métier, demandes de partage, mappings d'identités, tombstones et reçus, ainsi que les preuves et éditions Iceberg. Tester restauration et rejeu sans seconde publication ni second effet.
- Observabilité à préserver : santé/readiness, métriques Prometheus, parité des miroirs IAM, retard outbox/consommateurs, suppressions abandonnées, promotions échouées et divergence schéma physique/déclaré.

Séquence de migration

- Établir un inventaire par dataset : UUID, propriétaire IAM et sujet hérité, projet, nom technique, branches, table UUID Iceberg, schéma déclaré/physique, labels, droits, historique, tombstones et origine.
- Importer les métadonnées dans Catalog Rust et construire une projection parallèle en lecture. Comparer catalogue, schémas, détails, recherches, permissions de partage, compteurs et états sans écrire dans les dépôts sources.
- Formaliser les contrats de formats et de schéma ainsi que les scénarios des capacités DM-01 à DM-20. Traiter les conditions inversées/ambiguës par tickets, pas par copie silencieuse des comportements.
- Adopter les tables existantes et leur état retenu comme base migrée avec provenance déclarée. Ne pas fabriquer d'audit antérieur. Passer les nouvelles écritures par Writer/Audit/Publisher avec un seul propriétaire de publication.
- Basculer commandes et schedules par cohorte ; drainer l'outbox et les consommateurs existants. Garder les façades REST/SDK tant que les clients n'ont pas migré.

- Répéter restauration de Catalog depuis commandes et éditions, révocation pendant lecture, conflit après audit et drop échoué. Retirer Kotlin, Spring et dépendances Kafka/Keycloak Java une fois leur remplacement qualifié et le rollback de cohorte vérifié.

Écarts et décisions encore ouverts

- Aucun service ni test n'a été exécuté. Les comportements décrits sont des observations statiques à la révision donnée par le brief, et les acceptances sont des scénarios à implémenter ou exécuter.
- Quelle API publique doit porter diffusion, publication d'édition, promotion de sécurité et chiffrement pour que quatre opérations différentes ne partagent plus le même vocabulaire ?
- Comment adopter les datasets historiques dont schéma déclaré, ref physique et dernier refresh ne concordent pas ? L'outil de correction de data-serve n'est pas un historique d'éditions.
- Les limites artificielles de suppression de colonnes et la matrice de casts doivent-elles rester dans une façade historique ou évoluer dans le contrat Rust ?
- Comment retirer un lien INTERNAL accordé automatiquement via les routes actuelles dont /access/withdraw ne traite que ACCEPTED ?
- La politique Datasphere doit-elle migrer dans IAM Rust ou rester un adaptateur externe sans JVM ? Son résultat doit couvrir tous les sujets avant de permettre un partage.
- Quelles données de refresh/configuration/logs ont une rétention métier et quelles projections peut-on reconstruire ? Aucun checkpoint publié ne doit devenir dépendant de leur sauvegarde.
- Les collections, le lignage par édition et les kinds autres que la table doivent encore recevoir des API et des règles d'autorisation détaillées. Ils ne sont pas présents dans les contrôleurs examinés.

Références de l'analyse

[monorepo](#) · [services/data-model/AGENTS.md:1](#) Périmètre et consignes du composant.

[monorepo](#) · [services/data-model/app/gradle/libs.versions.toml:14](#) Versions Kotlin et Spring Boot déclarées.

[monorepo](#) · [services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/codemigration/runner/CleyropMigrationRunner.kt:37](#) Le code actuel fait échouer le démarrage sur migration ratée.

[monorepo](#) · [services/data-model/chart/values.yaml:60](#) Isolation réseau et identité gateway.

[monorepo](#) · [services/data-model/chart/values.yaml:224](#) Contrat de persistance et headers.

[monorepo](#) · [services/data-model/app/data-model-app/src/main/kotlin/com/cleyrop/data/model/app/common/outbox/OutboxRelay.kt:25](#) Publication au moins une fois et rétention.